

Assignment

Sub-queries & CTEs

Q1. Scalar Sub-query — Above Average CGPA

Write a query that lists full_name and cgpa of students whose CGPA is greater than the overall average

CGPA of all students. Use a scalar sub-query in the WHERE clause.

```
89 SELECT student_name, cgpa
90 FROM academics.students
91 WHERE cgpa > (
92     SELECT AVG(cgpa)
93     FROM academics.students
94 );
```

	student_name character varying (100)	cgpa numeric (4,2)
1	Priya Verma	9.12
2	Reetiks	8.65
3	Sunakshi	8.90

Q2. IN Sub-query — Students with A+ Grade

List the full_name of all students who have received at least one 'A+' grade in any course. Use a sub-query with IN.

```
97 SELECT student_name
98 FROM academics.students
99 WHERE student_id IN (
100     SELECT student_id
101     FROM academics.enrollments
102     WHERE grade = 'A+'
103 );
```

	student_name character varying (100)
1	Reetiks
2	Vanshika

Q3. NOT EXISTS — Students with No Enrollments

Using NOT EXISTS (not LEFT JOIN), find all students who have never enrolled in any course. Return student_id and full_name.

```
106 SELECT student_id, student_name
107 FROM academics.students s
108 WHERE NOT EXISTS (
109     SELECT 1
110     FROM academics.enrollments e
111     WHERE e.student_id = s.student_id
112 );
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

	student_id [PK] integer	student_name character varying (100)
1	9999	Test Student
2	2001	Rahul Sharma

Q4. Correlated Sub-query — Topper per Department

For each department, find the student(s) with the highest CGPA in that department. Return full_name,

dept_name, and cgpa. Use a correlated sub-query.

```
115 SELECT s.student_name, d.dept_name, s.cgpa
116 FROM academics.students s
117 JOIN academics.departments d
118     ON s.dept_id = d.dept_id
119 WHERE s.cgpa = (
120     SELECT MAX(s2.cgpa)
121     FROM academics.students s2
122     WHERE s2.dept_id = s.dept_id
123 );
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	student_name character varying (100)	dept_name character varying (100)	cgpa numeric (4,2)
1	Priya Verma	Computer Science	9.12

Q5. Sub-query in FROM (Derived Table)

Write a query that first calculates the average CGPA per department (as a derived table), then returns only those departments whose average CGPA is above 7.0. Show dept_name and avg_cgpa.

```
126 SELECT dept_name, avg_cgpa
127 FROM (
128     SELECT d.dept_name, AVG(s.cgpa) AS avg_cgpa
129     FROM academics.departments d
130     JOIN academics.students s
131         ON d.dept_id = s.dept_id
132     GROUP BY d.dept_name
133 ) AS dept_average
134 WHERE avg_cgpa > 7.0;
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	dept_name character varying (100)	avg_cgpa numeric
1	Computer Science	8.6400000000000000

Q6. EXISTS — Courses Taken by Specific Department

List all courses that have at least one enrollment from a student belonging to the 'Computer Science' department (or whichever dept_name exists in your data). Use EXISTS.

```
137 SELECT c.course_id, c.course_name
138 FROM academics.courses c
139 WHERE EXISTS (
140     SELECT 1
141     FROM academics.enrollments e
142     JOIN academics.students s
143         ON e.student_id = s.student_id
144     JOIN academics.departments d
145         ON s.dept_id = d.dept_id
146     WHERE e.course_id = c.course_id
147         AND d.dept_name = 'Computer Science'
148 );
```

Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1 of 1

	course_id [PK] integer	course_name character varying (100)
1	2	Database Management System...
2	3	Computer Networks
3	4	Operating Systems
4	100	Database

Q7. Basic CTE — High Achievers Report

Using a CTE named high_achievers (students with cgpa >= 8.0), list their full_name, the courses they enrolled in, and the grade. Order by full_name, course_name.

-- Write your WITH ... SELECT statement here:

```
151 WITH high_achievers AS (  
152     SELECT student_id, student_name  
153     FROM academics.students  
154     WHERE cgpa >= 8.0  
155 )  
156 SELECT h.student_name, c.course_name, e.grade  
157 FROM high_achievers h  
158 JOIN academics.enrollments e  
159     ON h.student_id = e.student_id  
160 JOIN academics.courses c  
161     ON e.course_id = c.course_id  
162 ORDER BY h.student_name, c.course_name;
```

Data Output Messages Notifications



Showing rows: 1 to 9 Page No: 1 of 1

	student_name character varying (100)	course_name character varying (100)	grade character (2)
1	Priya Verma	Database	B
2	Priya Verma	Database Management Syste...	B+
3	Priya Verma	Operating Systems	A
4	Reetiks	Computer Networks	B
5	Reetiks	Database	A
6	Reetiks	Database Management Syste...	A+
7	Sunakshi	Computer Networks	B+
8	Sunakshi	Database Management Syste...	A
9	Sunakshi	Operating Systems	A

Total rows: 9 Query complete 00:00:00.099

Q8. Multiple CTEs — Department Comparison

Create two CTEs:

- overall_avg → overall average CGPA of all students
- dept_avg → average CGPA per department

Then select departments whose average is higher than the overall average. Show dept_name, dept_avg, and overall_avg.

```

163
164
165 WITH overall_avg AS (
166     SELECT AVG(cgpa) AS overall_avg
167     FROM academics.students
168 ),
169 dept_avg AS (
170     SELECT d.dept_id, d.dept_name, AVG(s.cgpa) AS dept_avg
171     FROM academics.departments d
172     JOIN academics.students s
173         ON d.dept_id = s.dept_id
174     GROUP BY d.dept_id, d.dept_name
175 )
176
177 SELECT dept_name, dept_avg, overall_avg
178 FROM dept_avg
179 CROSS JOIN overall_avg
180 WHERE dept_avg > overall_avg;
181

```

Data Output
Messages
Notifications

+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 1

🔍

Page No: 1 of 1

⏪

⏴

⏵

⏩

	dept_name character varying (100) 🔒	dept_avg numeric 🔒	overall_avg numeric 🔒
1	Computer Science	8.6400000000000000	8.6120000000000000

Q9. CTE + Aggregation — Popular Courses

Using a CTE, find courses that have more than 2 enrollments. Return course_name and enrollment_count, ordered by enrollment_count descending.

```
183 WITH course_counts AS (  
184     SELECT course_id, COUNT(*) AS enrollment_count  
185     FROM academics.enrollments  
186     GROUP BY course_id  
187 )  
188 SELECT c.course_name, cc.enrollment_count  
189 FROM course_counts cc  
190 JOIN academics.courses c  
191     ON cc.course_id = c.course_id  
192 WHERE cc.enrollment_count > 2  
193 ORDER BY cc.enrollment_count DESC;
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	course_name character varying (100)	enrollment_count bigint
1	Database Management System	4
2	Computer Networks	3
3	Operating Systems	3

Q10. Correlated vs JOIN Rewrite

(a) Write a correlated sub-query that finds students whose CGPA is above their own department's average.

(b) Rewrite the same logic using JOINS (no correlated sub-query).

(c) In 1-2 sentences, which version do you prefer and why?

(a) Correlated Sub-query

```
197 SELECT s.student_name, s.cgpa  
198 FROM academics.students s  
199 WHERE s.cgpa > (  
200     SELECT AVG(s2.cgpa)  
201     FROM academics.students s2  
202     WHERE s2.dept_id = s.dept_id  
203 );
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1

	student_name character varying (100)	cgpa numeric (4,2)
1	Priya Verma	9.12
2	Reetiks	8.65
3	Sunakshi	8.90

(b) JOIN Version

```
206 SELECT s.student_name, s.cgpa
207 FROM academics.students s
208 JOIN (
209     SELECT dept_id, AVG(cgpa) AS avg_cgpa
210     FROM academics.students
211     GROUP BY dept_id
212 ) d
213 ON s.dept_id = d.dept_id
214 WHERE s.cgpa > d.avg_cgpa;
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	student_name character varying (100)	cgpa numeric (4,2)
1	Priya Verma	9.12
2	Reetiks	8.65
3	Sunakshi	8.90

(c) Which version do you prefer and why?

I prefer the JOIN version because it calculates the department averages once and is easier to understand when working with larger datasets.

Q11. Recursive CTE — Employee Hierarchy Levels

Using the employees table, write a recursive CTE that shows every employee, their managers name's (or NULL), and their hierarchy level (1 for top-level, 2 for direct reports, etc.). Order by level, emp_name.

```
218 WITH RECURSIVE employee_hierarchy AS (  
219     SELECT  
220         e.emp_id,  
221         e.emp_name,  
222         e.manager_id,  
223         1 AS hierarchy_level  
224     FROM academics.employees e  
225     WHERE e.manager_id IS NULL  
226  
227     UNION ALL  
228  
229     SELECT  
230         e.emp_id,  
231         e.emp_name,  
232         e.manager_id,  
233         eh.hierarchy_level + 1  
234     FROM academics.employees e  
235     JOIN employee_hierarchy eh  
236         ON e.manager_id = eh.emp_id  
237 )  
238 SELECT  
239     eh.emp_name,  
240     m.emp_name AS manager_name,  
241     eh.hierarchy_level  
242 FROM employee_hierarchy eh  
243 LEFT JOIN academics.employees m  
244     ON eh.manager_id = m.emp_id  
245 ORDER BY eh.hierarchy_level, eh.emp_name;
```

Data Output Messages Notifications

	emp_name character varying (100)	manager_name character varying (100)	hierarchy_level integer
1	Dr. Principal	[null]	1
2	Raj	[null]	1
3	Aman	Dr. Principal	2
4	Dr. Sharma	Dr. Principal	2
5	Priya	Dr. Principal	2
6	Dr. Kapoor	Dr. Sharma	3
7	Karan	Dr. Sharma	3
8	Neha	Dr. Sharma	3
9	Simran	Dr. Kapoor	4

Q12. Complex Multi-step Report

Build a report that answers: Which students scored an A or A+ in a course that belongs to a different department than their own, & Use CTEs or sub-queries as needed. Return: student_name, student_dept, course_name, course_dept, grade.

```
251 WITH student_courses AS (  
252     SELECT  
253         s.student_name AS student_name,  
254         sd.dept_name AS student_dept,  
255         c.course_name,  
256         cd.dept_name AS course_dept,  
257         e.grade  
258     FROM academics.students s  
259     JOIN academics.departments sd  
260         ON s.dept_id = sd.dept_id  
261     JOIN academics.enrollments e  
262         ON s.student_id = e.student_id  
263     JOIN academics.courses c  
264         ON e.course_id = c.course_id  
265     JOIN academics.departments cd  
266         ON c.dept_id = cd.dept_id  
267 )  
268 SELECT  
269     student_name,  
270     student_dept,  
271     course_name,  
272     course_dept,  
273     grade  
274 FROM student_courses  
275 WHERE grade IN ('A', 'A+')  
276 AND student_dept <> course_dept;
```

Data Output Messages Notifications

	student_name character varying (100)	student_dept character varying (100)	course_name character varying (100)	course_dept character varying (100)	grade character (2)
1	Sunakshi	Computer Science	Operating Systems	Mechanical Engineering	A
2	Reetiks	Computer Science	Database	Mechanical Engineering	A
3	Priya Verma	Computer Science	Operating Systems	Mechanical Engineering	A
4	Vanshika	Computer Science	Operating Systems	Mechanical Engineering	A+